UNIVERSIDAD DEL QUINDÍO · INGENIERÍA DE SISTEMAS Y COMPUTACIÓN · PROGRAMACIÓN III

Sistema Distribuido de Gestión de Emergencias

Arquitectura Concurrente en Elixir (OTP GenServer + Procesos Nativos), Algoritmos Recursivos, Persistencia en Texto Plano e Interfaz Web en Tiempo Real con Phoenix LiveView

Docente: Julián Esteban Gutiérrez Posada **Integrantes:** Juan Manuel Rojas · Luisa Fernanda Salazar

Stack: Elixir 1.18 / OTP 27 + Phoenix LiveView **Pruebas:** ExUnit (7/7 tests aprobados)

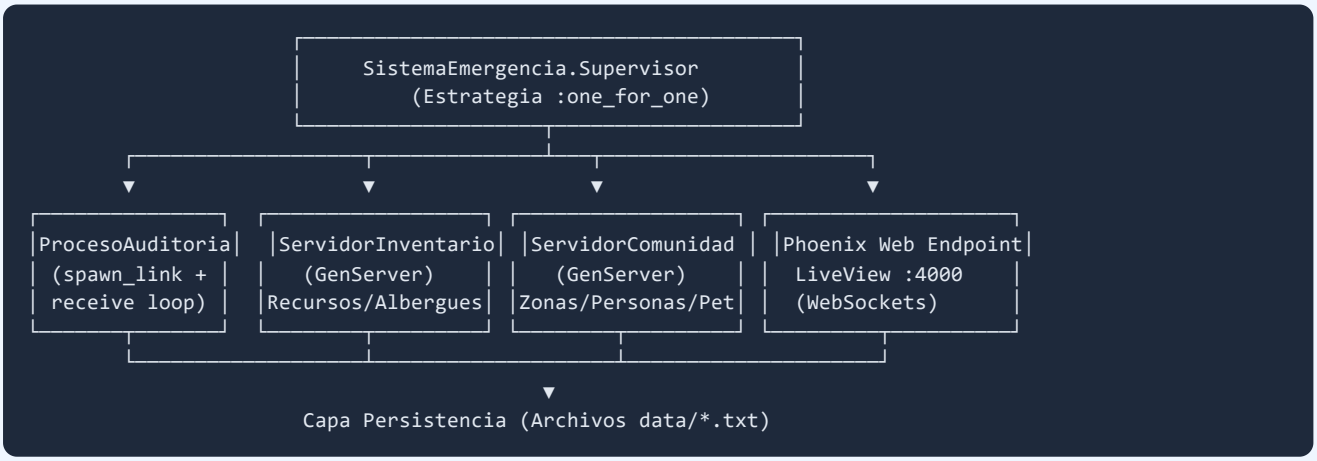
1. Matriz de Cumplimiento de Requisitos del Proyecto

Requisito de la Guía	Módulo / Archivo	Implementación y Solución Técnica
1. Usuarios, Roles y Auditoría (Sección 1.4.1)	`Modelos.Usuario` `ServidorComunidad` `ProcesoAuditoria`	4 roles (`:ciudadano` , `:voluntario` , `:coordinador` , `:administrador`) validados con pattern matching en `Modelos.autorizar_accion/2` . Auditoría concurrente con proceso nativo (`spawn_link` + `receive`).
2. Jerarquía Anidada de Zonas (Sección 1.4.3)	`Modelos.Zona` `Modelos.Sector` `Modelos.PuntoAtencion`	Estructura jerárquica de 3 niveles (`Zona → Sectores → Puntos de Atención`) recorrida recursivamente por `Recursividad.estadisticas_zona/1` .
3. Gestión de Recursos y Necesidades + Concurrencia (Secciones 1.4.4, 1.4.5 y 1.4.9)	`ServidorInventario` `Modelos.Recurso` `Modelos.Necesidad`	Asignación atómica en `GenServer` (`ejecutar_asignacion_atomica/4`). Resuelve el conflicto concurrente cuando 2 procesos piden 30L y 25L con solo 50L en stock.
4. Albergues y Control con MapSet (Sección 1.4.8)	`Modelos.Albergue` `ServidorInventario`	Usa `MapSet` en `personas_alojadas` para impedir ingresos duplicados y valida que nunca se supere `capacidad_maxima` .
5. Personas y Mensajes Familiares en Tiempo Real (Sección 1.4.6)	`ServidorComunidad` `CLI` / `EmergenciaLive`	Envío y entrega de mensajes con notificaciones automáticas en tiempo real entre terminales (`loop_monitor_mensajes/2`) y navegador web.

Requisito de la Guía	Módulo / Archivo	Implementación y Solución Técnica
6. Mascotas y Algoritmo de Coincidencias (Sección 1.4.7)	`Recursividad` `ServidorComunidad`	Cálculo recursivo de similitud sobre 100 puntos (especie +30, zona +25, color +20, tamaño +15, intersección de rasgos `MapSet` +10) filtrando puntaje ≥ 65.
7. Mínimo 4 Funciones Recursivas Propia (Requisito Funcional)	`SistemaEmergencia.Recursividad`	Se implementaron **5 funciones recursivas puras** sin depender de `Enum`: estadísticas jerárquicas, QuickSort de necesidades, match de mascotas, balance de historial y filtro genérico.
8. Persistencia en Archivos Planos (.txt) (Sección 1.4.9)	`SistemaEmergencia.Persistencia`	Almacenamiento y carga automática en `data/*.txt` (`resources.txt`, `needs.txt`, `shelters.txt`, `users.txt`, `zones.txt`, `people.txt`, `family_messages.txt`, `pets.txt`, `activity_log.txt`).
9. Interfaz Web Opcional en Phoenix (Requisito Opcional)	`SistemaEmergenciaWeb.Endpoint` `SistemaEmergenciaWeb.EmergenciaLive`	Dashboard interactivo en tiempo real con **Phoenix LiveView** en `http://localhost:4000` conectado a los mismos `GenServer` que la consola.

2. Arquitectura de Procesos y Árbol de Supervisión OTP

Árbol de Supervisión (`SistemaEmergencia.Application` - Estrategia `:one_for_one`):



3. Guion de Sustentación: Explicación Módulo por Módulo y Función por Función

3.1. Módulo `SistemaEmergencia.Modelos` (`lib/sistema_emergencia/modelos.ex`)

- **Propósito:** Define el modelo de dominio inmutable mediante 10 estructuras (`defstruct`) y funciones puras de validación con *guards* y *pattern matching*.
- **Funciones clave:**
  - `validar_id/1`, `validar_cantidad_positiva/1` y `validar_no_negativo/1`: usan guards (`when is_integer(n) and n > 0`) para garantizar que jamás ingresen cantidades negativas o vacías.
  - `autorizar_accion/2`: verifica permisos según el rol del usuario conectado (`:ciudadano`, `:voluntario`, `:coordinador`, `:administrador`).
  - `transicion_necesidad_valida?/2`: implementa la máquina de estados de una necesidad, impidiendo que una necesidad en estado `:atendida` o `:cancelada` vuelva a modificarse.
  - `peso_prioridad/1`: asigna pesos numéricos (`:critica → 4`, `:alta → 3`, `:media → 2`, `:baja → 1`) para el ordenamiento recursivo.

3.2. Módulo `SistemaEmergencia.Recursividad` (`lib/sistema_emergencia/recursividad.ex`)

- **Propósito:** Contiene los **5 algoritmos recursivos propios** (con caso base y llamada recursiva explícita, sin usar `Enum`):
  1. `estadisticas_zona/1` (con `recorrer_sectores/5` y `recorrer_puntos/3`): recorre la jerarquía `Zona → Sectores → Puntos de Atención` calculando total de sectores, total de puntos, capacidad instalada y afectados acumulados.
  2. `ordenar_necesidades_por_prioridad/1` (con `particionar_necesidades/4`): algoritmo **QuickSort funcional puro** que ordena las necesidades desde prioridad `:critica` hasta `:baja`.
  3. `buscar_coincidencias_mascotas/2` (con `comparar_perdidas/3` y `evaluar_contra_encontradas/3`): compara cada mascota perdida activa contra todas las encontradas activas y calcula el puntaje sobre 100 puntos (especie +30, zona +25, color +20, tamaño +15, intersección de `MapSet` +10), retornando las parejas con **puntaje ≥ 65**.
  4. `balance_historial_movimientos/1` (con `calcular_balance/3`): recorre la lista de movimientos de un recurso sumando entradas, salidas y saldo neto.
  5. `filtrar_recursivo/2` (con `do_filtrar/3` e `invertir_lista/2`): filtro recursivo de cola reutilizado en todo el sistema.

3.3. Módulo `SistemaEmergencia.Persistencia` (`lib/sistema_emergencia/persistencia.ex`)

- **Propósito:** Persiste las colecciones en texto plano dentro de la carpeta `data/`.
- **Funciones clave:**
  - `guardar_coleccion/2`: escribe cada struct en una línea legible dentro de su archivo `.txt`.
  - `cargar_coleccion/2` y `parsear_lineas/3`: lee el archivo plano y reconstruye recursivamente el mapa de estructuras en memoria.
  - `registrar_log_texto/3`: añade una línea con fecha UTC, usuario, acción y detalle en `data/activity_log.txt`.

3.4. Módulo `SistemaEmergencia.ProcesoAuditoria` (`lib/sistema_emergencia/proceso_auditoria.ex`)

- **Propósito:** Demuestra concurrencia nativa en Elixir mediante procesos básicos (`spawn_link`, `send` y `receive`).
- **Funciones clave:**
  - `start_link/1`: lanza el proceso con `spawn_link(fn -> loop([]) end)` y lo registra localmente como `:proceso_auditoria_emergencia`.
  - `registrar_evento/3`: envía un mensaje asíncrono con `send(pid, {:registrar, usuario, accion, detalle})` sin bloquear a los demás servidores.
  - `consultar_historial/0`: envía `{:consultar, self()}` y espera la respuesta con un bloque `receive ... after 2_000`.

3.5. Módulo `SistemaEmergencia.ServidorInventario` (`lib/sistema_emergencia/servidor_inventario.ex`)

- **Propósito:** `GenServer` que administra Recursos, Necesidades y Albergues.
- **Funciones clave:**
  - `ejecutar_asignacion_atomica/4`: verifica que la necesidad permita transición a `:asignada`, busca un recurso con stock suficiente y descuenta la cantidad en un solo paso atómico dentro del `handle_call` del `GenServer`. Así, cuando 2 procesos concurrentes piden 30L y 25L sobre un inventario de 50L, el primero deja 20L y el segundo recibe `{:error, :inventario_insuficiente}`.
  - `ejecutar_transferencia/5`: descuenta stock del centro de origen y crea/suma el lote en el centro de destino registrando ambos movimientos en el historial.
  - `cambiar_ocupacion_albergue/4`: gestiona entradas y salidas validando con `MapSet.member?` que un documento no se registre dos veces y que no se supere `capacidad_maxima`.

3.6. Módulo `SistemaEmergencia.ServidorComunidad` (`lib/sistema_emergencia/servidor_comunidad.ex`)

- **Propósito:** `GenServer` encargado de Usuarios, Zonas, Personas, Mensajes Familiares y Mascotas.
- **Funciones clave:**
  - `autenticar_o_registrar/5`: si el usuario no existe lo crea y conecta automáticamente; si ya existe valida su contraseña.
  - `construir_zona/5` y `anexar_sector/6`: construyen la jerarquía anidada `Zona → Sector → PuntoAtencion`.
  - `guardar_o_actualizar_persona/6`: registra una persona o actualiza su estado conservando la trazabilidad en `historial_estados`.
  - `construir_mensaje/4` y `marcar_mensaje_entregado/1`: permiten comunicación familiar en tiempo real con cambio de estado `:pendiente → :entregado`.
  - `cerrar_pareja_mascotas/3`: confirma el reencuentro entre una mascota perdida y una encontrada marcando ambos reportes como `:resuelta`.

3.7. Módulo `SistemaEmergencia.CLI` y `SistemaEmergenciaWeb.EmergenciaLive`

- **`SistemaEmergencia.CLI` (`cli.ex`):** provee el menú interactivo adaptado a cada rol (`loop_menu_usuario/1`), el proceso monitor en segundo plano que notifica mensajes nuevos en consola cada 1.2s (`iniciar_monitor_tiempo_real/1`), el intérprete de comandos de texto de la guía (`ejecutar_comando/1`) y las funciones de demostración (`ejecutar_flujo_ejemplo/0` y `simular_conflicto_concurrente/0`).

- `SistemaEmergenciaWeb.EmergenciaLive` (`emergencia_live.ex`)**:** vista interactiva en **Phoenix LiveView** (`http://localhost:4000`) que ofrece tarjetas KPI en vivo, formularios por pestañas, consola web de comandos y botones de simulación concurrente, sincronizándose automáticamente con las terminales abiertas.

4. Guía Paso a Paso de Ejecución (Terminales + Localhost)

Preparación Inicial (2 Terminales + Navegador Web funcionando en simultáneo):

1. Abre la **Terminal 1** en la carpeta `sistema_emergencia` y ejecuta: `iex -S mix`
   (Levanta los procesos OTP y el servidor web Phoenix en `http://localhost:4000`).
2. Abre la **Terminal 2** en la misma carpeta y ejecuta: `iex -S mix`
   (Detecta que el puerto 4000 ya está activo en la Terminal 1 e inicia la consola conectada a los mismos datos).
3. Abre tu navegador en `http://localhost:4000` y haz clic en **Reiniciar Estado** para iniciar la demostración desde cero.

PASO 1 Iniciar Sesión con Diferentes Roles en las 2 Terminales

- En **Terminal 1** y **Terminal 2** ejecuta: `SistemaEmergencia.menu()`
- **En Terminal 1 (Rol Ciudadano):** elige opción `1` → Usuario: `carlos` | Contraseña: `123` | Rol: `1` (Ciudadano) | Zona: `zona_norte`.
- **En Terminal 2 (Rol Coordinador):** elige opción `1` → Usuario: `laura` | Contraseña: `123` | Rol: `3` (Coordinador) | Zona: `centro`.

PASO 2 Registrar Zona, Sector y Consultar Estadísticas Recursivas (Terminal 2 - Laura)

- Opción `1` (**Registrar zona afectada**): Código: `centro` | Nombre: `Zona Centro` | Nivel: `critico` | Afectados: `100`.
- Opción `2` (**Agregar sector y puntos de atención**): Zona: `centro` | ID Sector: `SEC_2` | Nombre: `Barrio San Jose` | Afectados: `50` | ID Punto: `PTO_2` | Nombre Punto: `Escuela Central` | Capacidad: `80`.
- Opción `3` (**Consultar estado y estadísticas recursivas**): Código: `centro`. Muestra cómo `Recursividad.estadisticas_zona/1` recorre los 3 niveles y devuelve 2 sectores, 2 puntos, 180 de capacidad y 200 afectados acumulados.

PASO 3 Necesidades, QuickSort Recursivo, Recursos, Asignación y Transferencia

- **En Terminal 1 (Carlos - Ciudadano):** Opción `1` (Registrar necesidad) → Tipo: `agua` | Cantidad: `20` | Prioridad: `media` | Zona: `centro` | Descripción: `Agua familias`.
- **En Terminal 2 (Laura - Coordinadora):** Opción `5` (Registrar necesidad) → Tipo: `agua` | Cantidad: `30` | Prioridad: `critica` | Zona: `centro` | Descripción: `Hospital campaña`.
- **En Terminal 2:** Opción `8` (Listar necesidades ordenadas por prioridad) → Presiona Enter. Muestra que `N002` (`:critica`) sale primero que `N001` (`:media`) gracias al QuickSort recursivo.
- **En Terminal 2:** Opción `4` (Registrar recurso) → Código: `R001` | Nombre: `agua` | Categoría: `vital` | Cantidad: `100` | Unidad: `litros` | Ubicación: `acopio_1`.
- **En Terminal 2:** Opción `6` (Asignar recurso a necesidad) → ID: `N002` | Recurso: `agua` | Cantidad: `30`.
- **En Terminal 2:** Opción `7` (Transferir recurso) → Código: `R001` | Origen: `acopio_1` | Destino: `albergue_norte` | Cantidad: `20`.

PASO 4 Albergues, Personas y Mensajes Familiares en Tiempo Real (Terminal 1 ↔ Terminal 2 ↔ Web)

- **En Terminal 2 (Laura):** Opción `10` (Registrar albergue) → Código: `A01` | Nombre: `Albergue Norte` | Zona: `zona_norte` | Capacidad: `50`.
- **En Terminal 1 (Carlos):** Opción `3` (Reportar persona) → Documento: `1094` | Nombre: `Andres Perez` | Edad: `28` | Zona: `zona_norte` | Estado: `localizada`.
- **En Terminal 2 (Laura):** Opción `11` (Albergues) → opción `a` (Ingreso) → Albergue: `A01` | Documento: `1094`.
- **En Terminal 1 (Carlos):** Opción `5` (Enviar mensaje familiar) → Destinatario: `laura` | Mensaje: `Andres 1094 ya esta en el albergue norte`.
  → *En 1 segundo salta la alerta* 🔔 `[NOTIFICACIÓN EN TIEMPO REAL]` *en la Terminal 2 y en* `http://localhost:4000`.
- **En Terminal 2 (Laura):** Opción `12` (Familia) → opción `d` (Ver bandeja y marcar ENTREGADO) → Enter → Escribe `MSG_1`.
  → *En Terminal 1 y en* `http://localhost:4000` *aparece* ✅ `[ACTUALIZACIÓN EN TIEMPO REAL] :pendiente -> :entregado`.

PASO 5 Reporte de Mascotas y Algoritmo Recursivo `match_pets`

- **En Terminal 1 (Carlos):** Opción `7` (Reportar mascota) → Tipo: `perdida` | Especie: `perro` | Nombre: `Luna` | Color: `negra` | Tamaño: `mediano` | Zona: `zona_norte` | Contacto: `300111` | Características: `collar,mancha`.
- **En Terminal 2 (Laura):** Opción `13` → `a` (Reportar mascota) → Tipo: `encontrada` | Especie: `perro` | Nombre: `desconocido` | Color: `negra` | Tamaño: `mediano` | Zona: `zona_norte` | Contacto: `310999` | Características: `collar`.
- **En Terminal 2 (Laura):** Opción `13` → `b` (Buscar coincidencias) → Muestra el match entre `PET_001` y `PET_002` con **100 puntos**.
- **En Terminal 2 (Laura):** Opción `13` → `c` (Confirmar coincidencia) → `PET_001` y `PET_002` pasan a estado `:resuelta`.

PASO 6 Demostración en Vivo desde `http://localhost:4000` (Phoenix LiveView)

- Muestra en el navegador cómo todo lo realizado en las terminales ya aparece reflejado en las 6 pestañas.
- Haz clic en el botón amarillo ⚡ **Simular Conflicto Concurrente (2 Procesos)**: muestra el panel con las 2 `Task.async` compitiendo por 50L de agua (una asigna 30L dejando 20L en stock, y la otra recibe `{:error, :inventario_insuficiente}`).
- Haz clic en el botón verde ▶ **Cargar Flujo Ejemplo PDF** para ejecutar toda la guía de la página 7 con un clic.
- Abre la pestaña **6. Consola CLI & Auditoría**, presiona `summary` y muestra la bitácora del proceso nativo `ProcesoAuditoria`.

5. Anexo: Código Fuente Completo de la Solución

5.1. `lib/sistema_emergencia/application.ex`

```
defmodule SistemaEmergencia.Application do
  @moduledoc """
  Árbol de Supervisión Principal (OTP Tree - Requisito 1.8).
  Supervisa bajo estrategia `:one_for_one`:
    1. `ProcesoAuditoria` (Proceso independiente nativo con receive loop)
    2. `ServidorInventario` (GenServer de Recursos, Necesidades y Albergues)
    3. `ServidorComunidad` (GenServer de Usuarios, Zonas, Personas y Mascotas)
  """

  use Application

  @impl true
  def start(_type, _args) do
    base_children = [
      {Phoenix.PubSub, name: SistemaEmergencia.PubSub},
      SistemaEmergencia.ProcesoAuditoria,
      SistemaEmergencia.ServidorInventario,
      SistemaEmergencia.ServidorComunidad
    ]

    children =
      if iniciar_endpoint_web?() do
        base_children ++ [SistemaEmergenciaWeb.Endpoint]
      else
        IO.puts("[*] Phoenix ya está activo en http://localhost:4000 en otra terminal. Iniciando consola...")
        base_children
      end

    opts = [strategy: :one_for_one, name: SistemaEmergencia.Supervisor]
    Supervisor.start_link(children, opts)
  end

  defp iniciar_endpoint_web? do
    if Mix.env() == :test do
      true
    else
      case :gen_tcp.listen(4000, [:binary, active: false, ip: {127, 0, 0, 1}]) do
        {:ok, socket} ->
          :gen_tcp.close(socket)
          true

        {:error, :eaddrinuse} ->
          false

        _ ->
          true
      end
    end
  end
end
```

5.2. `lib/sistema_emergencia/modelos.ex`

```
defmodule SistemaEmergencia.Modelos do
  @moduledoc """
  Capa de Dominio: Define todas las estructuras inmutables (Structs),
  tipos válidos y funciones puras de validación.
```

```
  """

  defmodule Usuario do
    defstruct [:username, :password, :nombre, :rol, :zona, conectado: false]
  end

  # Jerarquía anidada: Zona -> Sectores -> Puntos de Atención (Requisito 1.4.3)
  defmodule PuntoAtencion do
    defstruct [:id, :nombre, :capacidad]
  end
  defmodule Sector do
    defstruct [:id, :nombre, :afectados_estimados, puntos: []]
  end
  defmodule Zona do
    defstruct [
      :codigo,
      :nombre,
      :nivel, # :critico | :alto | :medio | :bajo
      :afectados_estimados,
      sectores: [],
      albergues_ids: MapSet.new(),
      necesidades_ids: MapSet.new(),
      recursos_ids: MapSet.new(),
      reportes_ids: MapSet.new()
    ]
  end
  defmodule Necesidad do
    defstruct [
      :id,
      :tipo,        # :agua, :alimentos, :medicamentos, :primeros_auxilios, :refugio, :transporte, :rescate,
:otra
      :descripcion,
      :cantidad,
      :cantidad_asignada,
      :zona,
      :prioridad,   # :critica | :alta | :media | :baja
      :responsable,
      estado: :pendiente # :pendiente | :asignada | :en_atencion | :atendida | :cancelada
    ]
  end
  defmodule Recurso do
    defstruct [
      :codigo,
      :nombre,
      :categoria,
      :cantidad,
      :unidad,
      :ubicacion,
      historial: [] # Lista de tuplas: [{fecha, :entrada | :salida | :transferencia, cantidad, detalle}]
    ]
  end
  defmodule Persona do
    defstruct [
      :id,
      :documento,
      :nombre,
      :edad,
      :zona,
      estado: :reportada, # :sin_informacion | :reportada | :localizada | :trasladada | :en_albergue |
:reunificada
      historial_estados: [] # Lista de tuplas: [{fecha, estado_anterior, estado_nuevo}]
    ]
  end
  defmodule MensajeFamiliar do
    defstruct [
      :id,
      :documento_destinatario,
      :remitente,
```

```
      :contenido,
      :fecha,
      estado: :pendiente # :pendiente | :entregado
    ]
  end
  defmodule ReporteMascota do
    defstruct [
      :id,
      :tipo,            # :perdida | :encontrada
      :especie,         # "perro", "gato", etc.
      :nombre,
      :color,
      :tamano,          # "pequeno", "mediano", "grande"
      :caracteristicas, # MapSet de palabras clave para comparar similitud
      :zona,
      :contacto,
      estado: :activa,  # :activa | :posible_coincidencia | :resuelta
      pareja_id: nil
    ]
  end
  defmodule Albergue do
    defstruct [
      :codigo,
      :nombre,
      :zona,
      :capacidad_maxima,
      ocupacion_actual: 0,
      personas_alojadas: MapSet.new(), # Documentos únicos sin repetirse
      recursos_ids: MapSet.new(),
      necesidades_ids: MapSet.new()
    ]
  end

  @roles_validos [:ciudadano, :voluntario, :coordinador, :administrador]
  @niveles_zona [:critico, :alto, :medio, :bajo]
  @prioridades [:critica, :alta, :media, :baja]
  @estados_persona [:sin_informacion, :reportada, :localizada, :trasladada, :en_albergue, :reunificada]

  @doc "Valida que un texto o identificador no esté vacío."
  def validar_id(id) when is_binary(id) and byte_size(id) > 0, do: {:ok, String.trim(id)}
  def validar_id(_), do: {:error, :identificador_invalido}

  @doc "Valida usando guards que una cantidad o capacidad sea un entero positivo."
  def validar_cantidad_positiva(n) when is_integer(n) and n > 0, do: {:ok, n}
  def validar_cantidad_positiva(_), do: {:error, :cantidad_debe_ser_positiva}

  @doc "Valida usando guards que un valor no sea negativo (>= 0)."
  def validar_no_negativo(n) when is_integer(n) and n >= 0, do: {:ok, n}
  def validar_no_negativo(_), do: {:error, :cantidad_negativa_no_permitida}

  @doc "Valida roles de usuario."
  def validar_rol(rol) when rol in @roles_validos, do: {:ok, rol}
  def validar_rol(_), do: {:error, :rol_invalido}

  @doc "Valida permisos según el tipo de usuario (Requisito 1.4.1)."
  def autorizar_accion(%Usuario{rol: :administrador}, _accion), do: :ok
  def autorizar_accion(%Usuario{rol: :coordinador}, _accion), do: :ok
  def autorizar_accion(%Usuario{rol: :voluntario}, accion)
      when accion in [:registrar_necesidad, :registrar_recurso, :reportar_persona, :reportar_mascota,
:ingreso_albergue],
      do: :ok
  def autorizar_accion(%Usuario{rol: :ciudadano}, accion)
      when accion in [:registrar_necesidad, :reportar_persona, :enviar_mensaje, :reportar_mascota],
      do: :ok
  def autorizar_accion(_usuario, _accion), do: {:error, :permiso_denegado_para_rol}

  @doc "Valida niveles de afectación de una zona."
  def validar_nivel_zona(nivel) when nivel in @niveles_zona, do: {:ok, nivel}
```

```
  def validar_nivel_zona(_), do: {:error, :nivel_zona_invalido}

  @doc "Valida prioridad de una necesidad."
  def validar_prioridad(p) when p in @prioridades, do: {:ok, p}
  def validar_prioridad(_), do: {:error, :prioridad_invalida}

  @doc "Valida estado de persona afectada."
  def validar_estado_persona(e) when e in @estados_persona, do: {:ok, e}
  def validar_estado_persona(_), do: {:error, :estado_persona_invalido}

  @doc """
  Valida transiciones de estado para una Necesidad (Requisito 1.4.9).
  Impide que una necesidad ya :atendida o :cancelada vuelva a modificarse o consumir recursos.
  """
  def transicion_necesidad_valida?(:atendida, _nuevo), do: {:error, :necesidad_ya_atendida}
  def transicion_necesidad_valida?(:cancelada, _nuevo), do: {:error, :necesidad_ya_cancelada}
  def transicion_necesidad_valida?(:pendiente, nuevo) when nuevo in [:asignada, :en_atencion, :atendida,
:cancelada], do: :ok
  def transicion_necesidad_valida?(:asignada, nuevo) when nuevo in [:en_atencion, :atendida, :cancelada], do:
:ok
  def transicion_necesidad_valida?(:en_atencion, nuevo) when nuevo in [:atendida, :cancelada], do: :ok
  def transicion_necesidad_valida?(_, _), do: {:error, :transicion_estado_invalida}

  @doc """
  Convierte una prioridad en un peso numérico para poder ordenarla recursivamente.
  """
  def peso_prioridad(:critica), do: 4
  def peso_prioridad(:alta), do: 3
  def peso_prioridad(:media), do: 2
  def peso_prioridad(:baja), do: 1
  def peso_prioridad(_), do: 0
end
```

5.3. `lib/sistema_emergencia/recursividad.ex`

```
defmodule SistemaEmergencia.Recursividad do
  @moduledoc """
  Capa de Algoritmos Recursivos Propios (Sin depender de Enum).
  Cumple con el requisito de tener al menos 4 funciones recursivas sobre
  listas y estructuras jerárquicas.
  """
  alias SistemaEmergencia.Modelos
  alias SistemaEmergencia.Modelos.{Zona, Sector, PuntoAtencion, Necesidad, ReporteMascota}

  @doc """
  Calcula las estadísticas acumuladas de una Zona recorriendo recursivamente
  sus Sectores y, dentro de cada sector, sus Puntos de Atención.
  Retorna un mapa con: total de sectores, total de puntos, capacidad total en
  puntos de atención y total estimado de personas afectadas.
  """
  def estadisticas_zona(%Zona{afectados_estimados: base_afectados, sectores: sectores}) do
    {num_sectores, num_puntos, cap_puntos, afectados_sectores} =
      recorrer_sectores(sectores, 0, 0, 0, 0)
    %{
      total_sectores: num_sectores,
      total_puntos_atencion: num_puntos,
      capacidad_puntos_atencion: cap_puntos,
      afectados_acumulados: (base_afectados || 0) + afectados_sectores
    }
  end

  # Caso base de sectores
  defp recorrer_sectores([], acc_sec, acc_pts, acc_cap, acc_afec) do
    {acc_sec, acc_pts, acc_cap, acc_afec}
  end
```

```
  # Caso recursivo de sectores: procesa la cabeza y baja al nivel de puntos de atención
  defp recorrer_sectores(
         [%Sector{afectados_estimados: afec, puntos: puntos} | resto_sectores],
         acc_sec,
         acc_pts,
         acc_cap,
         acc_afec
       ) do
    {pts_sector, cap_sector} = recorrer_puntos(puntos, 0, 0)
    recorrer_sectores(
      resto_sectores,
      acc_sec + 1,
      acc_pts + pts_sector,
      acc_cap + cap_sector,
      acc_afec + (afec || 0)
    )
  end

  # Caso base de puntos de atención
  defp recorrer_puntos([], acc_pts, acc_cap), do: {acc_pts, acc_cap}

  # Caso recursivo de puntos de atención
  defp recorrer_puntos([%PuntoAtencion{capacidad: cap} | resto_puntos], acc_pts, acc_cap) do
    recorrer_puntos(resto_puntos, acc_pts + 1, acc_cap + (cap || 0))
  end

  @doc """
  Ordena recursivamente una lista de Necesidades de mayor a menor prioridad
  (:critica > :alta > :media > :baja) utilizando QuickSort funcional puro.
  """
  def ordenar_necesidades_por_prioridad([]), do: []
  def ordenar_necesidades_por_prioridad([pivote | resto]) do
    peso_pivote = Modelos.peso_prioridad(pivote.prioridad)
    {mayores_o_iguales, menores} = particionar_necesidades(resto, peso_pivote, [], [])
    ordenar_necesidades_por_prioridad(mayores_o_iguales) ++
      [pivote] ++
      ordenar_necesidades_por_prioridad(menores)
  end
  defp particionar_necesidades([], _peso_pivote, mayores, menores) do
    {invertir_lista(mayores, []), invertir_lista(menores, [])}
  end
  defp particionar_necesidades([%Necesidad{} = actual | resto], peso_pivote, mayores, menores) do
    if Modelos.peso_prioridad(actual.prioridad) >= peso_pivote do
      particionar_necesidades(resto, peso_pivote, [actual | mayores], menores)
    else
      particionar_necesidades(resto, peso_pivote, mayores, [actual | menores])
    end
  end

  # Función auxiliar recursiva de cola para invertir listas
  defp invertir_lista([], acc), do: acc
  defp invertir_lista([h | t], acc), do: invertir_lista(t, [h | acc])

  @doc """
  Busca recursivamente posibles coincidencias entre reportes de mascotas
  perdidas (:perdida) y encontradas (:encontrada) que sigan activas.
  Criterio de puntaje (máximo 100 puntos):
    - Misma especie (obligatorio): +30 pts
    - Misma zona: +25 pts
    - Mismo color: +20 pts
    - Mismo tamaño: +15 pts
    - Características en común (MapSet.intersection): +10 pts
  Se considera posible coincidencia si el puntaje es >= 65.
  """
  def buscar_coincidencias_mascotas(perdidas, encontradas) do
    comparar_perdidas(perdidas, encontradas, [])
  end
```

```elixir
  # Caso base: no quedan mascotas perdidas por comparar
  defp comparar_perdidas([], _encontradas, acc), do: invertir_lista(acc, [])

  # Caso recursivo: toma una mascota perdida y la compara contra todas las encontradas
  defp comparar_perdidas([%ReporteMascota{estado: :activa} = perdida | resto_perdidas], encontradas, acc) do
    coincidencias_para_esta = evaluar_contra_encontradas(perdida, encontradas, [])
    comparar_perdidas(resto_perdidas, encontradas, nuevas_coincidencias(coincidencias_para_esta, acc))
  end
  defp comparar_perdidas([_inactiva | resto_perdidas], encontradas, acc) do
    comparar_perdidas(resto_perdidas, encontradas, acc)
  end
  defp evaluar_contra_encontradas(_perdida, [], acc), do: acc
  defp evaluar_contra_encontradas(
         perdida,
         [%ReporteMascota{estado: :activa} = encontrada | resto_enc],
         acc
       ) do
    puntaje = calcular_puntaje_similitud(perdida, encontrada)
    if puntaje >= 65 do
      coincidencia = %{
        perdida_id: perdida.id,
        encontrada_id: encontrada.id,
        nombre_perdida: perdida.nombre,
        especie: perdida.especie,
        zona: perdida.zona,
        puntaje: puntaje
      }
      evaluar_contra_encontradas(perdida, resto_enc, [coincidencia | acc])
    else
      evaluar_contra_encontradas(perdida, resto_enc, acc)
    end
  end
  defp evaluar_contra_encontradas(perdida, [_inactiva | resto_enc], acc) do
    evaluar_contra_encontradas(perdida, resto_enc, acc)
  end
  defp calcular_puntaje_similitud(%ReporteMascota{} = p, %ReporteMascota{} = e) do
    if normalizar(p.especie) != normalizar(e.especie) do
      0
    else
      pts_especie = 30
      pts_zona = if normalizar(p.zona) == normalizar(e.zona), do: 25, else: 0
      pts_color = if normalizar(p.color) == normalizar(e.color), do: 20, else: 0
      pts_tamano = if normalizar(p.tamano) == normalizar(e.tamano), do: 15, else: 0
      comunes = MapSet.intersection(p.caracteristicas || MapSet.new(), e.caracteristicas || MapSet.new())
      pts_caract = if MapSet.size(comunes) > 0, do: 10, else: 0
      pts_especie + pts_zona + pts_color + pts_tamano + pts_caract
    end
  end
  defp normalizar(nil), do: ""
  defp normalizar(texto) when is_binary(texto), do: texto |> String.trim() |> String.downcase()
  defp normalizar(otro), do: to_string(otro) |> normalizar()
  defp nuevas_coincidencias([], acc), do: acc
  defp nuevas_coincidencias([h | t], acc), do: nuevas_coincidencias(t, [h | acc])

  @doc """
  Recorre recursivamente el historial de movimientos de un recurso
  (lista de tuplas `{fecha, tipo, cantidad, detalle}`) y calcula el total
  acumulado de entradas y salidas.
  """
  def balance_historial_movimientos(historial) do
    calcular_balance(historial, 0, 0)
  end
  defp calcular_balance([], entradas, salidas) do
    %{total_entradas: entradas, total_salidas: salidas, saldo_neto: entradas - salidas}
  end
  defp calcular_balance([{_fecha, :entrada, cant, _det} | resto], entradas, salidas) do
    calcular_balance(resto, entradas + cant, salidas)
  end
```

```
  defp calcular_balance([{_fecha, tipo, cant, _det} | resto], entradas, salidas)
       when tipo in [:salida, :asignacion, :transferencia_salida] do
    calcular_balance(resto, entradas, salidas + cant)
  end
  defp calcular_balance([_otro | resto], entradas, salidas) do
    calcular_balance(resto, entradas, salidas)
  end

  @doc """
  Función recursiva genérica para filtrar cualquier lista de elementos según
  un predicado sin usar Enum.filter/2.
  """
  def filtrar_recursivo(lista, funcion_predicado) do
    do_filtrar(lista, funcion_predicado, [])
  end
  defp do_filtrar([], _pred, acc), do: invertir_lista(acc, [])
  defp do_filtrar([h | t], pred, acc) do
    if pred.(h) do
      do_filtrar(t, pred, [h | acc])
    else
      do_filtrar(t, pred, acc)
    end
  end
end
```

5.4. `lib/sistema_emergencia/persistencia.ex`

```
defmodule SistemaEmergencia.Persistencia do
  @moduledoc """
  Capa de Persistencia en Archivos de Texto Plano (Requisito 1.4.9).
  Guarda y carga las estructuras del sistema en la carpeta `data/` en formato
  de texto plano legible (una entidad por línea en notación Elixir/texto),
  además de escribir el registro histórico de actividades en `activity_log.txt`.
  """
  @data_dir "data"
  @doc "Asegura que la carpeta `data/` exista."
  def inicializar_directorio do
    File.mkdir_p!(@data_dir)
  end

  @doc """
  Guarda un mapa o colección de entidades en un archivo `.txt` de texto plano.
  Cada registro se escribe en una línea legible usando `inspect/2`.
  """
  def guardar_coleccion(nombre_archivo, mapa_datos) when is_map(mapa_datos) do
    inicializar_directorio()
    ruta = Path.join(@data_dir, nombre_archivo)
    contenido =
      mapa_datos
      |> Map.values()
      |> Enum.map(&inspect(&1, limit: :infinity, printable_limit: :infinity))
      |> Enum.join("\n")
    contenido_final = if contenido == "", do: "", else: contenido <> "\n"
    File.write(ruta, contenido_final)
  end

  @doc """
  Lee un archivo `.txt` de la carpeta `data/` y reconstruye el mapa de entidades
  usando ` funcion_clave ` para indexar cada elemento por su ID/código.
  Si el archivo no existe aún, retorna un mapa vacío `%{}`.
  """
  def cargar_coleccion(nombre_archivo, funcion_clave) do
    inicializar_directorio()
    ruta = Path.join(@data_dir, nombre_archivo)
    case File.read(ruta) do
      {:ok, contenido} ->
```

```
        contenido
        |> String.split("\n", trim: true)
        |> parsear_lineas(funcion_clave, %{})
      {:error, :enoent} ->
        %{}
      {:error, _razon} ->
        %{}
    end
  end

  # Recorrido recursivo para convertir cada línea del archivo .txt en su Struct
  defp parsear_lineas([], _funcion_clave, acc), do: acc
  defp parsear_lineas([linea | resto], funcion_clave, acc) do
    try do
      {termino, _binding} = Code.eval_string(linea)
      clave = funcion_clave.(termino)
      parsear_lineas(resto, funcion_clave, Map.put(acc, clave, termino))
    rescue
      _ -> parsear_lineas(resto, funcion_clave, acc)
    end
  end

  @doc """
  Agrega una línea de texto plano al archivo `data/activity_log.txt`
  con marca de tiempo, usuario y acción realizada.
  """
  def registrar_log_texto(usuario, accion, detalle) do
    inicializar_directorio()
    ruta = Path.join(@data_dir, "activity_log.txt")
    fecha = DateTime.utc_now() |> Calendar.strftime("%Y-%m-%d %H:%M:%S")
    linea = "[#{fecha}] USUARIO=#{usuario} | ACCION=#{accion} | DETalle=#{detalle}\n"
    File.write(ruta, linea, [:append])
  end

  @doc "Lee todas las líneas del historial de actividad en texto plano."
  def leer_logs do
    ruta = Path.join(@data_dir, "activity_log.txt")
    case File.read(ruta) do
      {:ok, contenido} -> String.split(contenido, "\n", trim: true)
      {:error, _} -> []
    end
  end
end
```

5.5. `lib/sistema_emergencia/proceso_auditoria.ex`

```
defmodule SistemaEmergencia.ProcesoAuditoria do
  @moduledoc """
  Proceso Independiente Nativo (spawn_link + receive loop).
  Cumple con los Objetivos de Aprendizaje (1.3 y 1.4.1):
    - Aplicar concurrencia mediante procesos de Elixir y paso de mensajes (`send` / `receive`).
    - Registrar la actividad realizada por cada usuario en segundo plano.
  """
  alias SistemaEmergencia.Persistencia
  @nombre_proceso :proceso_auditoria_emergencia

  @doc """
  Especificación `child_spec/1` para que el Supervisor principal de la aplicación
  pueda supervisar y reiniciar automáticamente este proceso nativo si falla.
  """
  def child_spec(opts) do
    %{
      id: __MODULE__,
      start: {__MODULE__, :start_link, [opts]},
      type: :worker,
      restart: :permanent,
```

```
      shutdown: 500
    }
  end

  @doc "Inicia el proceso con `spawn_link` y lo registra bajo un nombre global local."
  def start_link(_opts \\ []) do
    pid = spawn_link(fn -> loop([]) end)
    Process.register(pid, @nombre_proceso)
    {:ok, pid}
  end

  @doc "Envía un mensaje asíncrono (`send`) al proceso de auditoría para registrar un evento."
  def registrar_evento(usuario, accion, detalle) do
    case Process.whereis(@nombre_proceso) do
      nil -> :error
      pid -> send(pid, {:registrar, usuario, accion, detalle})
    end
  end

  @doc "Solicita mediante paso de mensajes síncrono el historial en memoria del proceso."
  def consultar_historial do
    case Process.whereis(@nombre_proceso) do
      nil ->
        []
      pid ->
        send(pid, {:consultar, self()})
        receive do
          {:respuesta_historial, eventos} -> eventos
        after
          2_000 -> []
        end
    end
  end

  defp loop(eventos_memoria) do
    receive do
      {:registrar, usuario, accion, detalle} ->
        Persistencia.registrar_log_texto(usuario, accion, detalle)
        nuevo_evento = {DateTime.utc_now(), usuario, accion, detalle}
        loop([nuevo_evento | eventos_memoria])
      {:consultar, remitente_pid} ->
        send(remitente_pid, {:respuesta_historial, Enum.reverse(eventos_memoria)})
        loop(eventos_memoria)
      _otro ->
        loop(eventos_memoria)
    end
  end
end
```

5.6. `lib/sistema_emergencia/servidor_inventario.ex`

```
defmodule SistemaEmergencia.ServidorInventario do
  @moduledoc """
  GenServer encargado de Recursos, Necesidades y Albergues.
  Los callbacks `handle_call` son cortos y delegan la lógica a funciones puras privadas.
  """
  use GenServer

  alias SistemaEmergencia.Modelos
  alias SistemaEmergencia.Modelos.{Recurso, Necesidad, Albergue}
  alias SistemaEmergencia.{Persistencia, Recursividad, ProcesoAuditoria}

  @archivo_recursos "resources.txt"
  @archivo_necesidades "needs.txt"
  @archivo_albergues "shelters.txt"
```

```
  def start_link(opts \\ []), do: GenServer.start_link(__MODULE__, opts, name: __MODULE__)

  def registrar_recurso(codigo, nombre, categoria, cantidad, unidad, ubicacion, usuario \\ "sistema"),
    do: GenServer.call(__MODULE__, {:registrar_recurso, codigo, nombre, categoria, cantidad, unidad,
ubicacion, usuario})

  def listar_recursos(filtros \\ %{}),
    do: GenServer.call(__MODULE__, {:listar_recursos, filtros})

  def transferir_recurso(codigo, origen, destino, cantidad, usuario \\ "sistema"),
    do: GenServer.call(__MODULE__, {:transferir_recurso, codigo, origen, destino, cantidad, usuario})

  def registrar_necesidad(id, tipo, cantidad, prioridad, zona, descripcion, responsable \\ "sistema"),
    do: GenServer.call(__MODULE__, {:registrar_necesidad, id, tipo, cantidad, prioridad, zona, descripcion,
responsable})

  def listar_necesidades(filtros \\ %{}),
    do: GenServer.call(__MODULE__, {:listar_necesidades, filtros})

  def asignar_recurso(necesidad_id, criterio_recurso, cantidad, usuario \\ "sistema"),
    do: GenServer.call(__MODULE__, {:asignar_recurso, necesidad_id, criterio_recurso, cantidad, usuario})

  def registrar_albergue(codigo, nombre, zona, capacidad_maxima, usuario \\ "sistema"),
    do: GenServer.call(__MODULE__, {:registrar_albergue, codigo, nombre, zona, capacidad_maxima, usuario})

  def consultar_albergue(codigo),
    do: GenServer.call(__MODULE__, {:consultar_albergue, codigo})

  def registrar_ingreso_albergue(codigo, documento, usuario \\ "sistema"),
    do: GenServer.call(__MODULE__, {:ingreso_albergue, codigo, documento, usuario})

  def registrar_salida_albergue(codigo, documento, usuario \\ "sistema"),
    do: GenServer.call(__MODULE__, {:salida_albergue, codigo, documento, usuario})

  def reporte_albergues_mayor_ocupacion,
    do: GenServer.call(__MODULE__, :reporte_albergues)

  def recursos_criticos(umbral \\ 25),
    do: GenServer.call(__MODULE__, {:recursos_criticos, umbral})

  def reiniciar_estado,
    do: GenServer.call(__MODULE__, :reiniciar_estado)

  @impl true
  def init(_opts) do
    recursos = Persistencia.cargar_coleccion(@archivo_recursos, & &1.codigo)
    necesidades = Persistencia.cargar_coleccion(@archivo_necesidades, & &1.id)
    albergues = Persistencia.cargar_coleccion(@archivo_albergues, & &1.codigo)

    {:ok, %{recursos: recursos, necesidades: necesidades, albergues: albergues, contador_nec:
map_size(necesidades) + 1}}
  end

  @impl true
  def handle_call(:reiniciar_estado, _from, _state) do
    Enum.each([@archivo_recursos, @archivo_necesidades, @archivo_albergues],
&Persistencia.guardar_coleccion(&1, %{}))
    {:reply, :ok, %{recursos: %{}, necesidades: %{}, albergues: %{}, contador_nec: 1}}
  end

  @impl true
  def handle_call({:registrar_recurso, cod, nom, cat, cant, uni, ubi, user}, _from, state) do
    state = recargar_desde_disco(state)
    case construir_o_sumar_recurso(state.recursos, cod, nom, cat, cant, uni, ubi) do
      {:ok, recurso, nuevos_recursos} ->
        persistir_y_auditar(@archivo_recursos, nuevos_recursos, user, "register_resource", "#{cod} +#{cant}")
        {:reply, {:ok, recurso}, %{state | recursos: nuevos_recursos}}
```

```
      error ->
        {:reply, error, state}
    end
  end

  @impl true
  def handle_call({:listar_recursos, filtros}, _from, state) do
    state = recargar_desde_disco(state)
    {:reply, filtrar_recursos(Map.values(state.recursos), filtros), state}
  end

  @impl true
  def handle_call({:transferir_recurso, cod, origen, destino, cant, user}, _from, state) do
    state = recargar_desde_disco(state)
    case ejecutar_transferencia(state.recursos, cod, origen, destino, cant) do
      {:ok, resultado, nuevos_recursos} ->
        persistir_y_auditar(@archivo_recursos, nuevos_recursos, user, "transfer_resource", "#{cod}: #{cant} ->
#{destino}")
        {:reply, {:ok, resultado}, %{state | recursos: nuevos_recursos}}

      error ->
        {:reply, error, state}
    end
  end

  @impl true
  def handle_call({:registrar_necesidad, id_opt, tipo, cant, prio, zona, desc, resp}, _from, state) do
    state = recargar_desde_disco(state)
    id = generar_id_necesidad(id_opt, state.contador_nec)

    case construir_necesidad(state.necesidades, id, tipo, cant, prio, zona, desc, resp) do
      {:ok, necesidad, nuevas_nec} ->
        persistir_y_auditar(@archivo_necesidades, nuevas_nec, resp, "register_need", "#{id} (#{tipo})")
        {:reply, {:ok, necesidad}, %{state | necesidades: nuevas_nec, contador_nec: state.contador_nec + 1}}

      error ->
        {:reply, error, state}
    end
  end

  @impl true
  def handle_call({:listar_necesidades, filtros}, _from, state) do
    state = recargar_desde_disco(state)
    ordenadas =
      state.necesidades
      |> Map.values()
      |> filtrar_necesidades(filtros)
      |> Recursividad.ordenar_necesidades_por_prioridad()

    {:reply, ordenadas, state}
  end

  @impl true
  def handle_call({:asignar_recurso, nec_id, criterio_rec, cant, user}, _from, state) do
    state = recargar_desde_disco(state)
    case ejecutar_asignacion_atomica(state, nec_id, criterio_rec, cant) do
      {:ok, res, nuevo_state} ->
        Persistencia.guardar_coleccion(@archivo_recursos, nuevo_state.recursos)
        persistir_y_auditar(@archivo_necesidades, nuevo_state.necesidades, user, "assign_resource", "#{cant} -
> #{nec_id}")
        {:reply, {:ok, res}, nuevo_state}

      error ->
        {:reply, error, state}
    end
  end

  @impl true
```

```
  def handle_call({:registrar_albergue, cod, nom, zona, cap, user}, _from, state) do
    state = recargar_desde_disco(state)
    case construir_albergue(state.albergues, cod, nom, zona, cap) do
      {:ok, albergue, nuevos_alb} ->
        persistir_y_auditar(@archivo_albergues, nuevos_alb, user, "register_shelter", "#{cod} cap=#{cap}")
        {:reply, {:ok, albergue}, %{state | albergues: nuevos_alb}}

      error ->
        {:reply, error, state}
    end
  end

  @impl true
  def handle_call({:consultar_albergue, codigo}, _from, state) do
    state = recargar_desde_disco(state)
    case Map.get(state.albergues, codigo) do
      nil -> {:reply, {:error, :albergue_no_encontrado}, state}
      %Albergue{} = a -> {:reply, {:ok, %{albergue: a, cupos_disponibles: a.capacidad_maxima -
a.ocupacion_actual}}, state}
    end
  end

  @impl true
  def handle_call({:ingreso_albergue, codigo, doc, user}, _from, state) do
    state = recargar_desde_disco(state)
    case cambiar_ocupacion_albergue(state.albergues, codigo, doc, :ingreso) do
      {:ok, alb, nuevos_alb} ->
        persistir_y_auditar(@archivo_albergues, nuevos_alb, user, "register_entry", "#{doc} -> #{codigo}")
        {:reply, {:ok, alb}, %{state | albergues: nuevos_alb}}

      error ->
        {:reply, error, state}
    end
  end

  @impl true
  def handle_call({:salida_albergue, codigo, doc, user}, _from, state) do
    state = recargar_desde_disco(state)
    case cambiar_ocupacion_albergue(state.albergues, codigo, doc, :salida) do
      {:ok, alb, nuevos_alb} ->
        persistir_y_auditar(@archivo_albergues, nuevos_alb, user, "register_exit", "#{doc} <- #{codigo}")
        {:reply, {:ok, alb}, %{state | albergues: nuevos_alb}}

      error ->
        {:reply, error, state}
    end
  end

  @impl true
  def handle_call(:reporte_albergues, _from, state) do
    state = recargar_desde_disco(state)
    ordenados = Enum.sort_by(Map.values(state.albergues), &porcentaje_ocupacion/1, :desc)
    {:reply, ordenados, state}
  end

  @impl true
  def handle_call({:recursos_criticos, umbral}, _from, state) do
    state = recargar_desde_disco(state)
    criticos = Recursividad.filtrar_recursivo(Map.values(state.recursos), &(&1.cantidad <= umbral))
    {:reply, criticos, state}
  end

  defp construir_o_sumar_recurso(recursos, codigo, nombre, categoria, cantidad, unidad, ubicacion) do
    with {:ok, cod} <- Modelos.validar_id(codigo),
         {:ok, cant} <- Modelos.validar_no_negativo(cantidad) do
      movimiento = {fecha_actual(), :entrada, cant, "Ingreso en #{ubicacion}"}

      recurso =
```

```
        case Map.get(recursos, cod) do
          nil -> %Recurso{codigo: cod, nombre: norm(nombre), categoria: categoria, cantidad: cant, unidad:
unidad, ubicacion: ubicacion, historial: [movimiento]}
          %Recurso{} = r -> %Recurso{r | cantidad: r.cantidad + cant, historial: [movimiento | r.historial]}
        end

      {:ok, recurso, Map.put(recursos, cod, recurso)}
    end
  end

  defp ejecutar_transferencia(recursos, codigo, origen, destino, cantidad) do
    with {:ok, cant} <- Modelos.validar_cantidad_positiva(cantidad),
         %Recurso{} = rec <- Map.get(recursos, codigo) || {:error, :recurso_no_encontrado},
         true <- rec.ubicacion == origen || {:error, :ubicacion_origen_no_coincide},
         true <- rec.cantidad >= cant || {:error, :inventario_insuficiente} do
      cod_dest = "#{codigo}_#{destino}"
      rec_origen = %Recurso{rec | cantidad: rec.cantidad - cant, historial: [{fecha_actual(),
:transferencia_salida, cant, "A #{destino}"} | rec.historial]}
      {:ok, rec_destino, mapa_intermedio} = construir_o_sumar_recurso(recursos, cod_dest, rec.nombre,
rec.categoria, cant, rec.unidad, destino)

      {:ok, %{origen: rec_origen, destino: rec_destino}, Map.put(mapa_intermedio, codigo, rec_origen)}
    end
  end

  defp construir_necesidad(necesidades, id, tipo, cantidad, prioridad, zona, desc, resp) do
    with true <- not Map.has_key?(necesidades, id) || {:error, :id_necesidad_duplicado},
         {:ok, cant} <- Modelos.validar_cantidad_positiva(cantidad),
         {:ok, prio} <- Modelos.validar_prioridad(prioridad) do
      nueva = %Necesidad{id: id, tipo: norm(tipo), descripcion: desc, cantidad: cant, cantidad_asignada: 0,
zona: zona, prioridad: prio, responsable: resp}
      {:ok, nueva, Map.put(necesidades, id, nueva)}
    end
  end

  defp ejecutar_asignacion_atomica(state, nec_id, criterio_rec, cantidad) do
    with {:ok, cant} <- Modelos.validar_cantidad_positiva(cantidad),
         %Necesidad{} = nec <- Map.get(state.necesidades, nec_id) || {:error, :necesidad_no_encontrada},
         :ok <- Modelos.transicion_necesidad_valida?(nec.estado, :asignada),
         {:ok, %Recurso{} = rec} <- buscar_recurso_con_stock(state.recursos, criterio_rec, cant) do
      rec_act = %Recurso{rec | cantidad: rec.cantidad - cant, historial: [{fecha_actual(), :asignacion, cant,
"A #{nec.id}"} | rec.historial]}
      asignada = nec.cantidad_asignada + cant
      estado_nec = if asignada >= nec.cantidad, do: :atendida, else: :asignada
      nec_act = %Necesidad{nec | cantidad_asignada: asignada, estado: estado_nec}

      nuevo_state = %{state | recursos: Map.put(state.recursos, rec.codigo, rec_act), necesidades:
Map.put(state.necesidades, nec.id, nec_act)}
      {:ok, %{necesidad: nec_act, recurso: rec_act}, nuevo_state}
    end
  end

  defp construir_albergue(albergues, codigo, nombre, zona, capacidad) do
    with {:ok, cod} <- Modelos.validar_id(codigo),
         true <- not Map.has_key?(albergues, cod) || {:error, :albergue_ya_existe},
         {:ok, cap} <- Modelos.validar_cantidad_positiva(capacidad) do
      alb = %Albergue{codigo: cod, nombre: nombre, zona: zona, capacidad_maxima: cap}
      {:ok, alb, Map.put(albergues, cod, alb)}
    end
  end

  defp cambiar_ocupacion_albergue(albergues, codigo, doc, :ingreso) do
    with %Albergue{} = a <- Map.get(albergues, codigo) || {:error, :albergue_no_encontrado},
         true <- not MapSet.member?(a.personas_alojadas, doc) || {:error, :persona_ya_en_albergue},
         true <- a.ocupacion_actual < a.capacidad_maxima || {:error, :capacidad_maxima_superada} do
      act = %Albergue{a | ocupacion_actual: a.ocupacion_actual + 1, personas_alojadas:
MapSet.put(a.personas_alojadas, doc)}
      {:ok, act, Map.put(albergues, codigo, act)}
```

```
    end
  end

  defp cambiar_ocupacion_albergue(albergues, codigo, doc, :salida) do
    with %Albergue{} = a <- Map.get(albergues, codigo) || {:error, :albergue_no_encontrado},
         true <- MapSet.member?(a.personas_alojadas, doc) || {:error, :persona_no_estaba_en_albergue} do
      act = %Albergue{a | ocupacion_actual: max(0, a.ocupacion_actual - 1), personas_alojadas:
MapSet.delete(a.personas_alojadas, doc)}
      {:ok, act, Map.put(albergues, codigo, act)}
    end
  end

  defp buscar_recurso_con_stock(mapa_recursos, criterio, cantidad_req) do
    c = norm(criterio)
    candidatos = Recursividad.filtrar_recursivo(Map.values(mapa_recursos), &(norm(&1.codigo) == c or
norm(&1.nombre) == c))

    case candidatos do
      [] -> {:error, :recurso_no_encontrado}
      lista -> Enum.find(lista, &(&1.cantidad >= cantidad_req)) |> case do nil -> {:error,
:inventario_insuficiente}; r -> {:ok, r} end
    end
  end

  defp filtrar_recursos(lista, f) do
    Recursividad.filtrar_recursivo(lista, fn r ->
      (is_nil(f[:tipo]) or norm(r.nombre) == norm(f[:tipo])) and
        (is_nil(f[:ubicacion]) or norm(r.ubicacion) == norm(f[:ubicacion]))
    end)
  end

  defp filtrar_necesidades(lista, f) do
    Recursividad.filtrar_recursivo(lista, fn n ->
      (is_nil(f[:zona]) or norm(n.zona) == norm(f[:zona])) and
        (is_nil(f[:tipo]) or norm(n.tipo) == norm(f[:tipo])) and
        (is_nil(f[:prioridad]) or n.prioridad == f[:prioridad]) and
        (is_nil(f[:estado]) or n.estado == f[:estado])
    end)
  end

  defp persistir_y_auditar(archivo, datos, usuario, accion, detalle) do
    Persistencia.guardar_coleccion(archivo, datos)
    ProcesoAuditoria.registrar_evento(usuario, accion, detalle)
  end

  defp recargar_desde_disco(state) do
    recursos = Persistencia.cargar_coleccion(@archivo_recursos, & &1.codigo)
    necesidades = Persistencia.cargar_coleccion(@archivo_necesidades, & &1.id)
    albergues = Persistencia.cargar_coleccion(@archivo_albergues, & &1.codigo)
    %{state | recursos: recursos, necesidades: necesidades, albergues: albergues, contador_nec:
map_size(necesidades) + 1}
  end

  defp generar_id_necesidad(id, _) when is_binary(id) and byte_size(id) > 0, do: id
  defp generar_id_necesidad(_, cont), do: "N" <> String.pad_leading(Integer.to_string(cont), 3, "0")

  defp porcentaje_ocupacion(%Albergue{capacidad_maxima: c, ocupacion_actual: o}) when c > 0, do: o / c
  defp porcentaje_ocupacion(_), do: 0.0

  defp norm(v), do: v |> to_string() |> String.trim() |> String.downcase()
  defp fecha_actual, do: DateTime.utc_now() |> Calendar.strftime("%Y-%m-%d %H:%M:%S")
end
```

5.7. `lib/sistema_emergencia/servidor_comunidad.ex`

```
defmodule SistemaEmergencia.ServidorComunidad do
  @moduledoc """
  GenServer encargado de Usuarios, Zonas, Personas, Mensajes y Mascotas.
  Mantiene los `handle_call` breves delegando a funciones privadas pequeñas.
  """
  use GenServer

  alias SistemaEmergencia.Modelos
  alias SistemaEmergencia.Modelos.{Usuario, Zona, Sector, PuntoAtencion, Persona, MensajeFamiliar,
ReporteMascota}
  alias SistemaEmergencia.{Persistencia, Recursividad, ProcesoAuditoria}

  @archivo_usuarios "users.txt"
  @archivo_zonas "zones.txt"
  @archivo_personas "people.txt"
  @archivo_mensajes "family_messages.txt"
  @archivo_mascotas "pets.txt"

  def start_link(opts \\ []), do: GenServer.start_link(__MODULE__, opts, name: __MODULE__)

  def conectar_usuario(username, password, rol \\ :ciudadano, zona \\ "general"),
    do: GenServer.call(__MODULE__, {:conectar, username, password, rol, zona})

  def buscar_usuario(username),
    do: GenServer.call(__MODULE__, {:buscar_usuario, username})

  def desconectar_usuario(username \\ nil),
    do: GenServer.call(__MODULE__, {:desconectar, username})

  def usuario_actual, do: GenServer.call(__MODULE__, :usuario_actual)

  def registrar_zona(codigo, nombre, nivel, afectados \\ 0),
    do: GenServer.call(__MODULE__, {:registrar_zona, codigo, nombre, nivel, afectados})

  def agregar_sector_a_zona(codigo_zona, id_sector, nombre_sector, afectados, puntos \\ []),
    do: GenServer.call(__MODULE__, {:agregar_sector, codigo_zona, id_sector, nombre_sector, afectados,
puntos})

  def consultar_zona(codigo_zona), do: GenServer.call(__MODULE__, {:consultar_zona, codigo_zona})
  def listar_zonas, do: GenServer.call(__MODULE__, :listar_zonas)

  def reportar_persona(documento, nombre, zona, estado, edad \\ 30),
    do: GenServer.call(__MODULE__, {:reportar_persona, documento, nombre, zona, estado, edad})

  def buscar_persona(criterio), do: GenServer.call(__MODULE__, {:buscar_persona, criterio})
  def listar_personas, do: GenServer.call(__MODULE__, :listar_personas)

  def enviar_mensaje_familiar(doc_dest, contenido, remitente \\ "familiar"),
    do: GenServer.call(__MODULE__, {:enviar_mensaje, doc_dest, contenido, remitente})

  def listar_mensajes_familiares(doc_dest),
    do: GenServer.call(__MODULE__, {:listar_mensajes, doc_dest})

  def todos_los_mensajes,
    do: GenServer.call(__MODULE__, :todos_los_mensajes)

  def marcar_mensaje_entregado(id_mensaje),
    do: GenServer.call(__MODULE__, {:marcar_mensaje_entregado, id_mensaje})

  def reportar_mascota(tipo, especie, nombre, color, tamano, zona, contacto, caract \\ []),
    do: GenServer.call(__MODULE__, {:reportar_mascota, tipo, especie, nombre, color, tamano, zona, contacto,
caract})

  def buscar_coincidencias_mascotas, do: GenServer.call(__MODULE__, :match_pets)
```

```
  def confirmar_coincidencia_mascota(perdida_id, encontrada_id),
    do: GenServer.call(__MODULE__, {:confirmar_match, perdida_id, encontrada_id})

  def listar_mascotas(filtros \\ %{}), do: GenServer.call(__MODULE__, {:listar_mascotas, filtros})
  def reiniciar_estado, do: GenServer.call(__MODULE__, :reiniciar_estado)

  @impl true
  def init(_opts) do
    {:ok,
     %{
       usuarios: Persistencia.cargar_coleccion(@archivo_usuarios, & &1.username),
       usuario_sesion: nil,
       zonas: Persistencia.cargar_coleccion(@archivo_zonas, & &1.codigo),
       personas: Persistencia.cargar_coleccion(@archivo_personas, & &1.documento),
       mensajes: Persistencia.cargar_coleccion(@archivo_mensajes, & &1.id),
       mascotas: Persistencia.cargar_coleccion(@archivo_mascotas, & &1.id)
     }}
  end

  @impl true
  def handle_call(:reiniciar_estado, _from, _state) do
    [@archivo_usuarios, @archivo_zonas, @archivo_personas, @archivo_mensajes, @archivo_mascotas]
    |> Enum.each(&Persistencia.guardar_coleccion(&1, %{}))

    {:reply, :ok, %{usuarios: %{}, usuario_sesion: nil, zonas: %{}, personas: %{}, mensajes: %{}, mascotas: %
{}}}
  end

  @impl true
  def handle_call({:conectar, user, pass, rol, zona}, _from, state) do
    state = recargar_desde_disco(state)
    case autenticar_o_registrar(state.usuarios, user, pass, rol, zona) do
      {:ok, modo, usuario, nuevos_usuarios} ->
        persistir_y_auditar(@archivo_usuarios, nuevos_usuarios, usuario.username, "connect", "#{modo}")
        {:reply, {:ok, modo, usuario}, %{state | usuarios: nuevos_usuarios, usuario_sesion: usuario}}

      error ->
        {:reply, error, state}
    end
  end

  @impl true
  def handle_call({:desconectar, username}, _from, state) do
    state = recargar_desde_disco(state)
    target = username || (state.usuario_sesion && state.usuario_sesion.username)

    case cerrar_sesion_usuario(state.usuarios, target) do
      {:ok, desconectado, nuevos_usuarios} ->
        persistir_y_auditar(@archivo_usuarios, nuevos_usuarios, desconectado.username, "disconnect", "Cierre")
        {:reply, {:ok, desconectado}, %{state | usuarios: nuevos_usuarios, usuario_sesion: nil}}

      error ->
        {:reply, error, state}
    end
  end

  @impl true
  def handle_call(:usuario_actual, _from, state), do: {:reply, state.usuario_sesion, state}

  @impl true
  def handle_call({:buscar_usuario, username}, _from, state) do
    state = recargar_desde_disco(state)
    {:reply, Map.get(state.usuarios, String.trim(to_string(username))), state}
  end

  @impl true
  def handle_call({:registrar_zona, cod, nom, niv, afec}, _from, state) do
    state = recargar_desde_disco(state)
```

```
    case construir_zona(state.zonas, cod, nom, niv, afec) do
      {:ok, zona, nuevas_zonas} ->
        persistir_y_auditar(@archivo_zonas, nuevas_zonas, sesion(state), "register_zone", "#{cod} (#{niv})")
        {:reply, {:ok, zona}, %{state | zonas: nuevas_zonas}}

      error ->
        {:reply, error, state}
    end
  end

  @impl true
  def handle_call({:agregar_sector, cod_zona, id_sec, nom_sec, afec, puntos}, _from, state) do
    state = recargar_desde_disco(state)
    case anexar_sector(state.zonas, cod_zona, id_sec, nom_sec, afec, puntos) do
      {:ok, zona_act, nuevas_zonas} ->
        Persistencia.guardar_coleccion(@archivo_zonas, nuevas_zonas)
        {:reply, {:ok, zona_act}, %{state | zonas: nuevas_zonas}}

      error ->
        {:reply, error, state}
    end
  end

  @impl true
  def handle_call({:consultar_zona, cod_zona}, _from, state) do
    state = recargar_desde_disco(state)
    case Map.get(state.zonas, cod_zona) do
      nil -> {:reply, {:error, :zona_no_encontrada}, state}
      %Zona{} = z -> {:reply, {:ok, %{zona: z, estadisticas: Recursividad.estadisticas_zona(z)}}, state}
    end
  end

  @impl true
  def handle_call(:listar_zonas, _from, state) do
    state = recargar_desde_disco(state)
    {:reply, Map.values(state.zonas), state}
  end

  @impl true
  def handle_call({:reportar_persona, doc, nom, zona, est, edad}, _from, state) do
    state = recargar_desde_disco(state)
    case guardar_o_actualizar_persona(state.personas, doc, nom, zona, est, edad) do
      {:ok, persona, nuevas_personas} ->
        persistir_y_auditar(@archivo_personas, nuevas_personas, sesion(state), "report_person", "#{doc} -> #
{est}")
        {:reply, {:ok, persona}, %{state | personas: nuevas_personas}}

      error ->
        {:reply, error, state}
    end
  end

  @impl true
  def handle_call({:buscar_persona, criterio}, _from, state) do
    state = recargar_desde_disco(state)
    {:reply, filtrar_personas(Map.values(state.personas), criterio), state}
  end

  @impl true
  def handle_call(:listar_personas, _from, state) do
    state = recargar_desde_disco(state)
    {:reply, Map.values(state.personas), state}
  end

  @impl true
  def handle_call({:enviar_mensaje, doc_dest, contenido, remitente}, _from, state) do
    state = recargar_desde_disco(state)
    case construir_mensaje(state.mensajes, doc_dest, contenido, remitente) do
```

```
      {:ok, msg, nuevos_msgs} ->
        persistir_y_auditar(@archivo_mensajes, nuevos_msgs, sesion(state), "send_family_message", "Para #
{doc_dest}")
        {:reply, {:ok, msg}, %{state | mensajes: nuevos_msgs}}

      error ->
        {:reply, error, state}
    end
  end

  @impl true
  def handle_call({:listar_mensajes, doc_dest}, _from, state) do
    state = recargar_desde_disco(state)
    lista = Recursividad.filtrar_recursivo(Map.values(state.mensajes), &(&1.documento_destinatario ==
doc_dest))
    {:reply, lista, state}
  end

  @impl true
  def handle_call(:todos_los_mensajes, _from, state) do
    state = recargar_desde_disco(state)
    {:reply, state.mensajes, state}
  end

  @impl true
  def handle_call({:marcar_mensaje_entregado, id_msg}, _from, state) do
    state = recargar_desde_disco(state)
    case Map.get(state.mensajes, id_msg) do
      nil ->
        {:reply, {:error, :mensaje_no_encontrado}, state}

      %MensajeFamiliar{} = m ->
        act = %MensajeFamiliar{m | estado: :entregado}
        nuevos = Map.put(state.mensajes, id_msg, act)
        Persistencia.guardar_coleccion(@archivo_mensajes, nuevos)
        {:reply, {:ok, act}, %{state | mensajes: nuevos}}
    end
  end

  @impl true
  def handle_call({:reportar_mascota, tipo, esp, nom, col, tam, zona, cont, caract}, _from, state) do
    state = recargar_desde_disco(state)
    case construir_reporte_mascota(state.mascotas, tipo, esp, nom, col, tam, zona, cont, caract) do
      {:ok, rep, nuevas_mascotas} ->
        persistir_y_auditar(@archivo_mascotas, nuevas_mascotas, sesion(state), "report_pet", "#{rep.id} (#
{rep.tipo})")
        {:reply, {:ok, rep}, %{state | mascotas: nuevas_mascotas}}

      error ->
        {:reply, error, state}
    end
  end

  @impl true
  def handle_call(:match_pets, _from, state) do
    state = recargar_desde_disco(state)
    todas = Map.values(state.mascotas)
    perdidas = Recursividad.filtrar_recursivo(todas, &(&1.tipo == :perdida))
    encontradas = Recursividad.filtrar_recursivo(todas, &(&1.tipo == :encontrada))
    {:reply, Recursividad.buscar_coincidencias_mascotas(perdidas, encontradas), state}
  end

  @impl true
  def handle_call({:confirmar_match, p_id, e_id}, _from, state) do
    state = recargar_desde_disco(state)
    case cerrar_pareja_mascotas(state.mascotas, p_id, e_id) do
      {:ok, pareja, nuevas_mascotas} ->
        persistir_y_auditar(@archivo_mascotas, nuevas_mascotas, sesion(state), "confirm_pet_match", "#{p_id}<-
```

```
>#{e_id}")
        {:reply, {:ok, pareja}, %{state | mascotas: nuevas_mascotas}}

      error ->
        {:reply, error, state}
    end
  end

  @impl true
  def handle_call({:listar_mascotas, filtros}, _from, state) do
    state = recargar_desde_disco(state)
    {:reply, filtrar_mascotas(Map.values(state.mascotas), filtros), state}
  end

  defp autenticar_o_registrar(usuarios, username, password, rol, zona) do
    with {:ok, id} <- Modelos.validar_id(username),
         {:ok, r} <- Modelos.validar_rol(rol) do
      case Map.get(usuarios, id) do
        nil ->
          nuevo = %Usuario{username: id, password: password, nombre: id, rol: r, zona: zona, conectado: true}
          {:ok, :registrado_y_conectado, nuevo, Map.put(usuarios, id, nuevo)}

        %Usuario{password: ^password} = u ->
          conectado = %Usuario{u | conectado: true}
          {:ok, :conectado, conectado, Map.put(usuarios, id, conectado)}

        %Usuario{} ->
          {:error, :credenciales_invalidas}
      end
    end
  end

  defp cerrar_sesion_usuario(usuarios, target) do
    case target && Map.get(usuarios, target) do
      nil -> {:error, :usuario_no_conectado}
      %Usuario{} = u -> {:ok, %Usuario{u | conectado: false}, Map.put(usuarios, u.username, %Usuario{u |
conectado: false})}
    end
  end

  defp construir_zona(zonas, codigo, nombre, nivel, afectados) do
    with {:ok, cod} <- Modelos.validar_id(codigo),
         {:ok, niv} <- Modelos.validar_nivel_zona(nivel),
         {:ok, afec} <- Modelos.validar_no_negativo(afectados) do
      sector_base = %Sector{id: "#{cod}_S1", nombre: "Sector Base #{nombre}", afectados_estimados: div(afec,
2), puntos: [%PuntoAtencion{id: "#{cod}_P1", nombre: "Punto #{nombre}", capacidad: 100}]}
      zona = %Zona{codigo: cod, nombre: nombre, nivel: niv, afectados_estimados: afec, sectores:
[sector_base]}
      {:ok, zona, Map.put(zonas, cod, zona)}
    end
  end

  defp anexar_sector(zonas, cod_zona, id_sec, nom_sec, afec, lista_puntos) do
    case Map.get(zonas, cod_zona) do
      nil ->
        {:error, :zona_no_encontrada}

      %Zona{} = z ->
        pts = Enum.map(lista_puntos, fn {i, n, c} -> %PuntoAtencion{id: i, nombre: n, capacidad: c} end)
        sec = %Sector{id: id_sec, nombre: nom_sec, afectados_estimados: afec, puntos: pts}
        z_act = %Zona{z | sectores: z.sectores ++ [sec]}
        {:ok, z_act, Map.put(zonas, cod_zona, z_act)}
    end
  end

  defp guardar_o_actualizar_persona(personas, documento, nombre, zona, estado, edad) do
    with {:ok, doc} <- Modelos.validar_id(documento),
         {:ok, est} <- Modelos.validar_estado_persona(estado) do
```

```
      f = fecha_actual()

      p =
        case Map.get(personas, doc) do
          nil -> %Persona{id: "PER_#{doc}", documento: doc, nombre: nombre, edad: edad, zona: zona, estado:
est, historial_estados: [{f, :ninguno, est}]}
          %Persona{} = ex -> %Persona{ex | nombre: nombre, zona: zona, estado: est, historial_estados: [{f,
ex.estado, est} | ex.historial_estados]}
        end

      {:ok, p, Map.put(personas, doc, p)}
    end
  end

  defp construir_mensaje(mensajes, doc_dest, contenido, remitente) do
    with {:ok, doc} <- Modelos.validar_id(doc_dest),
         {:ok, txt} <- Modelos.validar_id(contenido) do
      id = "MSG_" <> Integer.to_string(map_size(mensajes) + 1)
      msg = %MensajeFamiliar{id: id, documento_destinatario: doc, remitente: remitente, contenido: txt, fecha:
fecha_actual()}
      {:ok, msg, Map.put(mensajes, id, msg)}
    end
  end

  defp construir_reporte_mascota(mascotas, tipo, esp, nom, col, tam, zona, cont, caract) do
    with {:ok, tipo_atom} <- normalizar_tipo_mascota(tipo) do
      id = "PET_" <> String.pad_leading(Integer.to_string(map_size(mascotas) + 1), 3, "0")
      set_c = caract |> List.wrap() |> Enum.map(&norm/1) |> MapSet.new()
      rep = %ReporteMascota{id: id, tipo: tipo_atom, especie: esp, nombre: nom, color: col, tamano: tam,
caracteristicas: set_c, zona: zona, contacto: cont}
      {:ok, rep, Map.put(mascotas, id, rep)}
    end
  end

  defp cerrar_pareja_mascotas(mascotas, p_id, e_id) do
    with %ReporteMascota{} = p <- Map.get(mascotas, p_id) || {:error, :reporte_no_encontrado},
         %ReporteMascota{} = e <- Map.get(mascotas, e_id) || {:error, :reporte_no_encontrado},
         true <- (p.estado != :resuelta and e.estado != :resuelta) || {:error, :reporte_ya_resuelto} do
      p_res = %ReporteMascota{p | estado: :resuelta, pareja_id: e.id}
      e_res = %ReporteMascota{e | estado: :resuelta, pareja_id: p.id}
      {:ok, {p_res, e_res}, mascotas |> Map.put(p.id, p_res) |> Map.put(e.id, e_res)}
    end
  end

  defp filtrar_personas(lista, criterio) do
    c = norm(criterio)
    Recursividad.filtrar_recursivo(lista, &(norm(&1.documento) == c or String.contains?(norm(&1.nombre), c) or
norm(&1.zona) == c or norm(&1.estado) == c))
  end

  defp filtrar_mascotas(lista, f) do
    Recursividad.filtrar_recursivo(lista, fn m ->
      (is_nil(f[:especie]) or norm(m.especie) == norm(f[:especie])) and
        (is_nil(f[:zona]) or norm(m.zona) == norm(f[:zona])) and
        (is_nil(f[:estado]) or m.estado == f[:estado])
    end)
  end

  defp normalizar_tipo_mascota(t) when t in [:perdida, "perdida", "lost"], do: {:ok, :perdida}
  defp normalizar_tipo_mascota(t) when t in [:encontrada, "encontrada", "found"], do: {:ok, :encontrada}
  defp normalizar_tipo_mascota(_), do: {:error, :tipo_reporte_mascota_invalido}

  defp persistir_y_auditar(archivo, datos, usuario, accion, detalle) do
    Persistencia.guardar_coleccion(archivo, datos)
    ProcesoAuditoria.registrar_evento(usuario, accion, detalle)
  end

  defp recargar_desde_disco(state) do
```

```
    %{
      state
      | usuarios: Persistencia.cargar_coleccion(@archivo_usuarios, & &1.username),
        zonas: Persistencia.cargar_coleccion(@archivo_zonas, & &1.codigo),
        personas: Persistencia.cargar_coleccion(@archivo_personas, & &1.documento),
        mensajes: Persistencia.cargar_coleccion(@archivo_mensajes, & &1.id),
        mascotas: Persistencia.cargar_coleccion(@archivo_mascotas, & &1.id)
    }
  end

  defp sesion(%{usuario_sesion: %Usuario{username: u}}), do: u
  defp sesion(_), do: "sistema"
  defp norm(v), do: v |> to_string() |> String.trim() |> String.downcase()
  defp fecha_actual, do: DateTime.utc_now() |> Calendar.strftime("%Y-%m-%d %H:%M:%S")
end
```

5.8. `lib/sistema_emergencia_web/endpoint.ex` y `router.ex`

```
defmodule SistemaEmergenciaWeb.Endpoint do
  @moduledoc """
  Endpoint HTTP y WebSocket de Phoenix para el Sistema Distribuido de Gestión de Emergencias.
  Expone la interfaz web interactiva en http://localhost:4000
  """
  use Phoenix.Endpoint, otp_app: :sistema_emergencia

  @session_options [
    store: :cookie,
    key: "_sistema_emergencia_key",
    signing_salt: "emergencia_sign_salt",
    same_site: "Lax"
  ]

  socket "/live", Phoenix.LiveView.Socket,
    websocket: [connect_info: [session: @session_options]]

  plug Plug.Static,
    at: "/assets/phoenix",
    from: :phoenix,
    only: ~w(phoenix.min.js phoenix.js)

  plug Plug.Static,
    at: "/assets/phoenix_live_view",
    from: :phoenix_live_view,
    only: ~w(phoenix_live_view.min.js phoenix_live_view.js)

  plug Plug.RequestId
  plug Plug.Parsers,
    parsers: [:urlencoded, :multipart, :json],
    pass: ["*/*"],
    json_decoder: Phoenix.json_library()

  plug Plug.MethodOverride
  plug Plug.Head
  plug Plug.Session, @session_options
  plug SistemaEmergenciaWeb.Router
end


defmodule SistemaEmergenciaWeb.Router do
  @moduledoc """
  Enrutador principal de Phoenix que dirige las peticiones HTTP y WebSocket
  hacia el panel interactivo en tiempo real (`EmergenciaLive`).
  """
  use Phoenix.Router
  import Phoenix.LiveView.Router
```

```
  pipeline :browser do
    plug :accepts, ["html"]
    plug :fetch_session
    plug :put_root_layout, html: {SistemaEmergenciaWeb.Layouts, :root}
    plug :protect_from_forgery
    plug :put_secure_browser_headers
  end

  scope "/", SistemaEmergenciaWeb do
    pipe_through :browser

    live "/", EmergenciaLive, :index
  end
end
```